

Architecture des Ordinateurs — Chapitres 2 à 5

Nom : BARETS

Prénom : Emma

ID Étudiant : 100672190DG

Table des matières

Chapitre 2 — La Représentation des Nombres Entiers Positifs	3
Chapitre 3 — Opérations en Binaire & Représentation des Nombres Signés	16
Chapitre 4 — Représentation des Nombres Flottants et des Caractères	24
Chapitre 5 - Algèbre de Boole	38

Chapitre 2 — La Représentation des Nombres Entiers Positifs

2.6 Exercices

Pour ce chapitre 2, vous devez rendre les exercices **2.3** et **2.12**

L'exercice **2.2**, qui vous demandera un peu de programmation, est pour un **Bonus**.

Exercice 2.2 (Bonus)

Décrire dans le détail, avec des phrases, la suite d'opérations qu'il faut effectuer pour additionner deux nombres écrits en chiffres romains. Écrire ensuite le programme correspondant dans le langage de votre choix (cf. Petit Guide).

Pour vous aider : Une manière "intelligemment-paresseuse" de procéder est d'effectuer l'addition en base dix des nombres saisis par l'utilisateur puis de convertir le résultat à l'aide du programme réalisé dans l'exercice 2.3.

Réponse :

Pour additionner deux nombres écrits en chiffres romains il faut d'abord demander à l'utilisateur d'entrer deux nombres en chiffres romains, par exemple X et V. Ensuite on va convertir chaque nombre romain en nombre entier à l'aide d'une fonction qui va associer chaque symbole romain à sa valeur et applique les règles de soustraction. Puis on additionne les deux entiers obtenus pour obtenir la somme en base 10. On convertit cette somme en chiffre romain (voir programme ex 2.3) et on affiche le résultat en chiffre romain.

(PS : J'ai également découvert `".upper()"` dans ce cours mais je ne sais pas si on peut l'utiliser donc j'ai préféré inclure dans "valeurs" les minuscules et les majuscules : <https://fr.python-3.com/?p=2788>)

Pour insérer du code Python dans ce fichier LaTeX, j'ai utilisé ce site pour comprendre comment faire : <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>.

Voici ce que j'ai fait :

```
1 def romain_vers_entier(romain):
```

Je définis une fonction qui prend en paramètre une chaîne de caractères représentant un nombre romain.

```
1 valeurs = {  
2     'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000,  
3     'i':1, 'v':5, 'x':10, 'l':50, 'c':100, 'd':500, 'm':1000  
4 }
```

Je crée un dictionnaire qui associe chaque symbole romain à sa valeur en base 10. J'y inclus aussi les minuscules parce qu'un utilisateur peut aussi taper xiv au lieu de XIV.

```
1 total = 0  
2 i = 0
```

J'initialise deux variables : total va accumuler la valeur au fur et à mesure qu'on lit les symboles et i est l'indice qui va me permettre de me déplacer dans la chaîne.

```
1 while i < len(romain):  
2     lettre = romain[i]
```

On entre dans la boucle principale. On dit que tant qu'il reste des caractères à lire, on continue. Je stocke le caractère actuel dans lettre juste pour que le code soit plus lisible.

```
1 if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
```

Pour les chiffres romains, quand un symbole est plus petit que celui qui le suit, on doit le soustraire plutôt que l'additionner. Par exemple IX = 10 - 1 = 9 ou CM = 1000 - 100 = 900. Donc ici, je compare la valeur du symbole courant avec celle du suivant. Le `i + 1 < len(romain)` dit que si on est sur le dernier caractère, il n'y a pas de "suivant" et donc on ne va pas essayer d'y accéder puisqu'il n'y a rien ensuite et ça provoquerait une erreur.

```
1 else:
2     total += valeurs[lettre]
3     i += 1
```

Ici, on dit que quand le symbole courant est plus grand ou égal au suivant on l'additionne simplement et on passe au suivant.

```
1 return total
```

Une fois toute la chaîne parcourue, on retourne le total obtenu.

```
1 romain1 = input("Entrez le premier nombre romain : ")
2 romain2 = input("Entrez le second nombre romain : ")
```

On demande à l'utilisateur de saisir ses deux nombres romains. `input()` retourne toujours une chaîne de caractères, ce qui tombe bien puisque c'est exactement ce qu'attend notre fonction.

```
1 entier1 = romain_vers_entier(romain1)
2 entier2 = romain_vers_entier(romain2)
```

On convertit chacun des deux nombres en entier en appelant la fonction qu'on vient de définir.

```
1 somme = entier1 + entier2
```

On additionne simplement les deux entiers obtenus après conversion. Le résultat est stocké dans la variable somme pour être affiché ensuite.

```
1 print(f"La somme en base 10 est : {somme}")
```

```

1 # Exercice 2.2 (Bonus)
2
3 def romain_vers_entier(romain):
4     """
5     Convertit un chiffre romain en entier.
6     Prend en compte les majuscules et les minuscules.
7     """
8     # Dictionnaire avec la valeur de chaque symbole romain
9     valeurs = {
10         'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000,
11         'i':1, 'v':5, 'x':10, 'l':50, 'c':100, 'd':500, 'm':1000
12     }
13
14     total = 0
15     i = 0
16
17     while i < len(romain):
18         lettre = romain[i]
19         if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
20             total += valeurs[romain[i + 1]] - valeurs[lettre]
21             i += 2
22         else:
23             total += valeurs[lettre]
24             i += 1
25
26     return total
27
28 romain1 = input("Entrez le premier nombre romain : ")
29 romain2 = input("Entrez le second nombre romain : ")
30
31 entier1 = romain_vers_entier(romain1)
32 entier2 = romain_vers_entier(romain2)
33 somme = entier1 + entier2
34
35 print(f"La somme en base 10 est : {somme}")

```

Quelques tests réalisés pour voir si le programme fonctionne bien :

Calcul de base :

```

>>> %Run 2_2_3.py
Entrez le premier nombre romain : V
Entrez le second nombre romain : V
La somme en base 10 est : 10

```

FIGURE 1 – Test 1

Si un symbole plus petit est avant le plus grand + test avec les minuscules

```
>>> %Run 2_2_3.py
Entrez le premier nombre romain : iv
Entrez le second nombre romain : v
La somme en base 10 est : 9
```

FIGURE 2 – Test 2

Voici une deuxième version du programme (Voir explication et test¹)

```
1 # Exercice 2.2 (Bonus)
2
3 def romain_vers_entier(romain):
4     """
5     Convertit un chiffre romain en entier.
6     Gere les majuscules et minuscules.
7     Applique les regles classiques :
8     - Pas plus de 3 repetitions consecutives pour I, X, C, M
9     - V, L, D ne sont jamais repetes
10    - Limite de 3999 pour le resultat
11    """
12
13    # Dictionnaire contenant la valeur de chaque symbole romain
14    valeurs = {
15        'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000,
16        'i':1, 'v':5, 'x':10, 'l':50, 'c':100, 'd':500, 'm':1000
17    }
18
19    # Lettres autorisees a etre repetees jusqu'a 3 fois
20    repetables = {'I', 'X', 'C', 'M', 'i', 'x', 'c', 'm'}
21
22    compteur = 1 # Compteur pour suivre le nombre de repetitions d'affilees
23    total = 0    # Somme finale en base 10
24    i = 0        # Index pour parcourir la chaine
25
26    # Parcours de chaque symbole du nombre romain
27    while i < len(romain):
28        lettre = romain[i]
29
30        # Verification que le caractere est bien un symbole romain
31        if lettre not in valeurs:
32            print(f"Symbole romain invalide detecte : {lettre}")
33            return None
34
35        # Verification des repetitions d'affilees
36        if i > 0 and romain[i] == romain[i-1]:
37            compteur += 1
38            # Si le symbole peut etre repete, on limite a 3
39            if lettre in repetables and compteur > 3:
40                print(f"Erreur : le symbole {lettre} ne peut pas etre repete plus de
41                    3 fois consecutivement.")
42                return None
43            # Si le symbole ne peut pas etre repete du tout (V, L, D)
44            elif lettre not in repetables:
```

```

44         print(f"Erreur : le symbole {lettre} ne peut pas etre repete.")
45         return None
46     else:
47         compteur = 1 # Reinitialisation du compteur si symbole different
48
49     # Verification que le total ne depasse pas 3999
50     if total > 3999:
51         print("Attention : la numeration romaine standard ne supporte pas les
52             nombres > 3999")
53         return None
54
55     # Dans le cas ou le premier nombre est plus petit que le deuxieme
56     if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
57         total += valeurs[romain[i + 1]] - valeurs[lettre]
58         i += 2 # On saute deux symboles car ils ont ete utilises ensemble
59     else:
60         total += valeurs[lettre]
61         i += 1 # On passe au symbole suivant
62
63     return total # Renvoie la valeur entiere obtenue

```

Conclusion

Cet exercice demandait d'additionner deux nombres romains en passant par une conversion en base 10. Pour ça, j'ai écrit une fonction qui parcourt chaque symbole du nombre romain, applique la règle de soustraction quand c'est nécessaire et retourne l'entier correspondant. Une fois les deux nombres convertis, l'addition se fait en une ligne et le résultat est affiché.

Ce que j'ai compris / appris

Cet exercice était relativement accessible, le code en lui-même ne m'a pas posé de difficultés majeures. Ce qui a été plus intéressant, c'est de réfléchir aux choix de conception. Par exemple, pour gérer les majuscules et les minuscules, j'ai choisi d'inclure les deux dans le dictionnaire, mais j'aurais pu faire autrement. Par exemple, Python propose `.upper()`, qui retourne une chaîne avec tous les caractères en majuscules ou `.lower()`, qui fait l'inverse. Les deux ignorent les chiffres et les symboles, ce qui les rend sûrs à utiliser dans ce contexte. J'ai préféré rester sur les dictionnaires comme ça mon code est plus explicite pour l'instant, mais ça peut être une piste d'amélioration.

Par ailleurs, je n'ai pas eu à gérer de vraies erreurs dans ce programme parce que les tests ont tous fonctionné du premier coup, mis à part un `}` oublié à un moment, ce qui n'est pas très instructif à mentionner. C'est pour ça que je me suis contenté de trois exemples de sorties comme demandé, sans capture d'écrans de gestion d'erreurs particulière.

Enfin, cet exercice m'a aussi confronté à une question que je me pose encore. En NSI au lycée, on n'avait pas le droit d'utiliser des fonctions déjà définies comme `sum()` sans les avoir recodées soi-même au préalable. À la fac, je ne sais pas encore exactement où se situe la limite. Dans le sens où est-ce qu'on peut utiliser librement ce que Python propose tant qu'on l'explique et qu'on cite ses sources ou est-ce qu'il faut tout reconstruire ? Pour l'instant j'ai fait le choix de rester prudent et de m'appuyer sur les bases mais je pense qu'en prenant mes marques je commencerai à explorer davantage.

Exercice 2.3 (A Rendre)

Réaliser un programme, dans le langage de votre choix (cf Petit Guide), qui imprime un nombre saisi par l'utilisateur en chiffres romains.

Pour vous aider : Voir les liens de la section "Pour aller (Encore) plus loin".

L'idée ici est l'inverse de l'exercice précédent. En effet, on part d'un entier et on veut obtenir sa représentation en chiffres romains.

Voici ce que j'ai fait :

```
1 def entier_vers_roman(nombre):
```

On définit une fonction qui prend un entier en paramètre et retourne sa représentation romaine sous forme de chaîne.

```
1 romains = [  
2     ('M',1000), ('CM',900), ('D',500), ('CD',400),  
3     ('C',100), ('XC',90), ('L',50), ('XL',40),  
4     ('X',10), ('IX',9), ('V',5), ('IV',4), ('I',1)  
5 ]
```

On crée une liste de tuples qui associe chaque symbole romain à sa valeur, triée du plus grand au plus petit. On pourrait utiliser un dictionnaire, mais ici une liste de tuples est plus adaptée. Un dictionnaire sert surtout à chercher une valeur par clé alors qu'ici on veut parcourir les symboles dans un ordre précis (du plus grand au plus petit). Aussi, une chose importante, on y inclut les cas de soustraction comme CM (900), CD (400), XC (90) etc. Je préfère les intégrer directement dans la liste comme ça j'évite d'avoir à gérer ces cas séparément.

```
1 resultat = ""
```

On initialise une chaîne vide qui va accumuler les symboles romains au fur et à mesure de la conversion. C'est l'équivalent du total = 0 dans la fonction précédente, mais pour une chaîne.

```
1 for symbole, valeur in romains:
```

On parcourt chaque paire (symbole, valeur) de la liste dans l'ordre.

```
1 while nombre >= valeur:
```

Pour chaque symbole, on vérifie en boucle si le nombre restant est encore supérieur ou égal à sa valeur. On utilise un while et non un if parce qu'un même symbole peut apparaître plusieurs fois. Par exemple 3000 = MMM donc on va soustraire 1000 trois fois de suite avant de passer au symbole suivant.

```
1 resultat += symbole  
2 nombre -= valeur
```

Ensuite, tant que la condition est vraie, on ajoute le symbole au résultat et on soustrait sa valeur au nombre restant. En sachant que on ne peut pas ajouter un symbole sans soustraire sa valeur sinon on bouclerait indéfiniment.

```
1 return resultat
```

Une fois tous les symboles parcourus et le nombre réduit à zéro, on retourne la chaîne construite.


```
1 romain_resultat = entier_vers_roman(somme)
```

On appelle la fonction sur la variable somme calculée dans l'exercice précédent.

```
1 print(f"La somme de {romain1} et {romain2} en chiffres romains est : {romain_resultat  
  }")
```

Voici le code final :

```
1 # Exercice 2.3 (A Rendre)
2
3 def entier_vers_roman(nombre):
4     """
5     Convertit un entier en chiffre romain.
6     """
7     # Liste des symboles romains et leur valeur associee (du plus grand au plus petit
8     # pour la conversion)
9     romains = [
10         ('M',1000), ('CM',900), ('D',500), ('CD',400),
11         ('C',100), ('XC',90), ('L',50), ('XL',40),
12         ('X',10), ('IX',9), ('V',5), ('IV',4), ('I',1)
13     ]
14
15     resultat = "" # Chaine pour la conversion finale
16
17     # Parcours de chaque symbole et sa valeur
18     for symbole, valeur in romains:
19         # Tant que le nombre restant est superieur ou egal a la valeur du symbole
20         while nombre >= valeur:
21             resultat += symbole # Ajouter le symbole romain au resultat
22             nombre -= valeur    # Soustraire cette valeur du nombre restant
23
24     return resultat # Resultat du chiffre romain
25
26 # Conversion de la somme obtenue avant en chiffre romain
27 romain_resultat = entier_vers_roman(somme)
28
29 # Affiche la somme des deux nombres saisis en chiffres romains
30 print(f"La somme de {romain1} et {romain2} en chiffres romains est : {romain_resultat  
  }")
```

Pour tester les programmes précédents, j'ai utilisé le site que vous nous avez conseillé dans "Pour aller plus loin" (WolframAlpha) : <https://www.wolframalpha.com/>

Exemple de test avec ce site :



FIGURE 3 – Test 1

```
>>> %Run 2_2_3.py

Entrez le premier nombre romain : XLVIII
Entrez le second nombre romain : LXXII
La somme en base 10 est : 120
```

FIGURE 4 – Test 2

Test pour voir si les deux programmes fonctionnent ensemble :

```
>>> %Run 2_2_3.py

Entrez le premier nombre romain : L
Entrez le second nombre romain : C
La somme en base 10 est : 150
La somme de L et C en chiffres romains est : CL
```

FIGURE 5 – Test 2

Explication : De plus, en lisant la page Wikipedia (le lien que vous nous avez fourni pour aller plus loin : https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine) je pense qu'on aurait aussi pu rajouter et modifier légèrement le programme, c'est pour cela que j'ai décidé de faire une v2.

On apprend que le système romain ne gère pas naturellement les fractions ou les nombres <1 , et que tout ce qu'on fait en chiffres romains concerne des entiers ≥ 1 (cela n'impactera pas ces deux programmes).

La numération romaine traditionnelle permet de représenter les nombres jusqu'à 3999. Par convention, le chiffre 4 s'écrit IV (5-1), 40 s'écrit XL et 400 s'écrit CD. Il n'existe pas de symbole officiel pour 5000, ce qui fixe une limite pratique pour cette numération. Cette limite est mentionnée par Georges Ifrah dans *Histoire universelle des chiffres* (tome 1, page 454).

Certaines discussions suggèrent que des variantes existaient : par exemple, des répétitions plus longues de symboles comme IIII pour 4 ou XXXX pour 40 ont pu être utilisées, mais il n'y a pas de preuve formelle que ce soit systématique.

Donc on aurait pu aussi rajouter une condition pour que les III s'arrête à 3 ou à 4.

Bien que certaines sources montrent qu'il serait possible de gérer des nombres supérieurs à 3999 ou d'utiliser des variations historiques, ces cas restent majoritairement des suppositions ou des exceptions. Pour cette raison, j'ai décidé de rajouter une condition pour que mon programme se limite aux nombres ≤ 3999 et également la règle que les lettres I, X, C et M ne peuvent pas apparaître plus de trois fois consécutivement, et que V, L et D ne peuvent jamais se répéter, afin de respecter les conventions classiques de la numération romaine.

Voici la V2 de la question 2.3 :

```
1 # Exercice 2.3 (A rendre)
2
3 def entier_vers_romain(nombre):
4
5     if nombre > 3999:
6         print("La numeration romaine standard ne supporte pas les nombres > 3999")
7         return None
8
9     # Liste des symboles romains et leur valeur associee (du plus grand au plus petit
10    )
11    romains = [
12        ('M',1000), ('CM',900), ('D',500), ('CD',400),
13        ('C',100), ('XC',90), ('L',50), ('XL',40),
14        ('X',10), ('IX',9), ('V',5), ('IV',4), ('I',1)
15    ]
16
17    resultat = "" # Chaine pour la conversion finale
18
19    # Parcours de chaque symbole et de sa valeur
20    for symbole, valeur in romains:
21        count = 0 # Compteur pour limiter les repetitions
22
23        # Boucle tant que le nombre reste >= valeur et que la repetition est
24        autorisee
25        while nombre >= valeur and (
26            (symbole in ['I','X','C','M'] and count < 3) or
27            (symbole in ['V','L','D'] and count < 1) or
28            (symbole in ['CM','CD','XC','XL','IX','IV']) # symboles composes sans
29            limite
30        ):
31            resultat += symbole
32            nombre -= valeur
33            count += 1
34
35    return resultat
```

```

29         nombre -= valeur
30         count += 1
31
32     return resultat # Retourne le chiffre romain
33
34 # Conversion de la somme en chiffre romain
35 romain_resultat = entier_vers_romain(somme)
36
37 # Affichage du resultat ou message d'impossibilite
38 if romain_resultat is not None:
39     print(f"La somme de {romain1} et {romain2} en chiffres romains est : {
40         romain_resultat}")
41 else:
42     print("Impossible de convertir ce nombre en chiffres romains standard.")

```

Pour les programmes finaux des exercices 2.2 et 2.3, j'ai inclus un fichier Python en annexe afin que vous puissiez les exécuter et les tester directement.

```

Entrez le premier nombre romain : IIII
Entrez le second nombre romain : I
Erreur : le symbole I ne peut pas être répété plus de 3 fois consécutivement.
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.

```

FIGURE 6 – Test 1 : Test en mettant plusieurs lettres d'affilées

```

Entrez le premier nombre romain : XXXXXXMMXXMMXXMMXXMMXXMMXXMMXXMMXXMMXXMMXX
Entrez le second nombre romain : X
Attention : la numération romaine standard ne supporte pas les nombres > 3999
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.

```

FIGURE 7 – Test 2 : Test en obtenant un nombre supérieur à 3999

```

Entrez le premier nombre romain : VV
Entrez le second nombre romain : V
Erreur : le symbole V ne peut pas être répété.
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.

```

FIGURE 8 – Test 3 : Test répétition lettre V

Conclusion

Cet exercice demandait de faire l'inverse de l'exercice précédent. Autrement dit, il faut prendre un entier et afficher sa représentation en chiffres romains. Pour ça, j'ai écrit une fonction qui parcourt une liste de symboles du plus grand au plus petit et qui soustrait chaque valeur au nombre restant jusqu'à arriver à zéro. J'ai aussi réalisé une V2 du programme après avoir lu les sources fournies dans le cours, qui ajoute des vérifications sur les règles classiques de la numération romaine comme la limite à 3999 et les restrictions sur les répétitions de certains symboles.

Ce que j'ai compris / appris

En lisant la page Wikipedia fournie dans le cours, j'ai découvert des choses que je ne savais pas sur la numération romaine : elle ne gère pas les fractions ni les nombres inférieurs à 1, elle est limitée par convention à 3999 car il n'existe pas de symbole officiel pour 5000 et certaines lettres comme V, L et D ne peuvent jamais être répétées, tandis que I, X, C et M sont limitées à trois répétitions consécutives. Ces règles m'ont semblé suffisamment importantes pour les intégrer dans une V2 du programme, avec des vérifications explicites.

De plus, j'ai compris pourquoi une liste de tuples est ici plus adaptée qu'un dictionnaire puisqu'on doit traiter les symboles du plus grand au plus petit et qu'un dictionnaire ne garantit pas cet ordre. Inclure directement les cas de soustraction comme CM ou XL dans la liste m'a aussi évité de gérer ces cas séparément, ce qui simplifie beaucoup l'algorithme.

Comme pour l'exercice précédent, je n'ai pas eu de gestion d'erreurs complexe à mettre en place, le programme a fonctionné tout au long du développement et les trois tests réalisés avec WolframAlpha ont tous donné les bons résultats.

Exercice 2.12 (A Rendre)

Convertir le nombre $(3391)_{10}$ en base -5. (Facile si vous avez déjà répondu au 2.11)

Rappel : les restes des divisions par -5 doivent être positifs. Vous ne devez pas utiliser la forme (par exemple) $18 = 3 \ 5 \ 3$ mais plutôt $18 = 4 \cdot 5 + 2$.

Réponse :

Pour convertir le nombre 3391 en base -5, on fait :

1) 3391 divisé par -5 = -678 avec un reste de 1.

On écrit donc : $3391 = (-678) \cdot (-5) + 1$

2) -678 divisé par -5 = 136 avec un reste de 2.

On écrit : $-678 = 136 \cdot (-5) + 2$

3) 136 divisé par -5 = -27 avec un reste de 1.

On écrit : $136 = (-27) \cdot (-5) + 1$

4) -27 divisé par -5 = 6 avec un reste de 3.

On écrit : $-27 = 6 \cdot (-5) + 3$

5) 6 divisé par -5 = -1 avec un reste de 1.

On écrit : $6 = (-1) \cdot (-5) + 1$

6) -1 divisé par -5 = 1 avec un reste de 4.

On écrit : $-1 = 1 \cdot (-5) + 4$

7) 1 divisé par -5 = 0 avec un reste de 1.

On écrit : $1 = 0 \cdot (-5) + 1$

Ensuite, on lit tous les restes de bas en haut pour obtenir le nombre en base -5.

Donc, $(3391)_{10} = (1413121)_{-5}$.

Conclusion

Cet exercice demandait de convertir le nombre 3391 en base -5. Le principe est le même que pour une conversion en base positive c'est-à-dire qu'on divise successivement par la base et on relève les restes à chaque étape jusqu'à obtenir un quotient nul. La seule différence ici est que la base est négative, ce qui demande quelques ajustements. Je me suis entraîné grâce aux exercices "Au Choix".

Ce que j'ai compris / appris

J'ai compris que :

- les restes des divisions par -5 doivent toujours être positifs, c'est-à-dire appartenir à $\{0, 1, 2, 3, 4\}$;
- quand le reste obtenu est négatif, il faut ajuster : par exemple, si on obtient $-1 = 3 \times (-5) - 3$, ce n'est pas valide. On réécrit plutôt $-1 = 1 \times (-5) + 4$, en ajustant le quotient en conséquence ;
- on arrête les divisions uniquement quand le quotient est égal à zéro et pas avant (j'avais justement oublié une étape dans mon premier calcul à cause de ça) ;
- une fois toutes les étapes effectuées, on lit les restes de bas en haut pour obtenir la représentation finale en base -5.

Conclusion générale

Ce chapitre portait sur la représentation des nombres entiers positifs. C'est un chapitre qui m'a semblé globalement accessible avec des notions qui se comprennent bien une fois qu'on a assimilé la logique de base. Les exercices sur les chiffres romains m'ont aussi permis d'apprendre des choses que je ne savais pas comme les règles de répétition des symboles ou la limite à 3999. Dans l'ensemble, c'est un chapitre assez intéressant qui m'a donné envie d'aller un peu plus loin, comme avec les V2 que j'ai réalisées.

Sources du chapitre 2

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- Python-3.com. (s. d.). *La méthode .upper()* <https://fr.python-3.com/?p=2788>
- BornToCode.fr. (2026). *LaTeX : comment insérer et customiser du code source* <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>
- WolframAlpha. (s. d.). *WolframAlpha : Computational intelligence* <https://www.wolframalpha.com/>
- Wikipédia. (2023). *Discussion : Numération romaine* https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine
- Ifrah, G. (1998). *Histoire universelle des chiffres* (tome 1, p. 454). Robert Laffont.

Chapitre 3 — Opérations en Binaire & Représentation des Nombres Signés

3.5 Une première Série d'Exercices

Dans cette première série d'exercice, nous vous invitons à les réaliser sur au brouillon. Ce sont des exercices d'entraînement. Attention, l'exercice 3.6 sera à rendre (voir liste plus loin).

Exercice 3.6 (A Rendre)

La base -2 n'utilise que les chiffres 0 et 1. (Facile) Traduire en décimal les seize valeurs entre $(0000)_{-2}$ et $(1111)_{-2}$. (Plus difficile) Quelle est la table d'addition de la base -2 ?

Pour vous aider : voir l'article "Système négabinaire" sur Wikipédia...

Réponse :

Selon le cours, pour convertir un nombre écrit en base négative (ici base -2) en décimal, on applique la même méthode que pour toute base : chaque chiffre (ou bit) est multiplié par la puissance correspondante de la base, en commençant par la droite avec la puissance 0. La valeur décimale du nombre est ensuite la somme de toutes ces valeurs.

Pour la base -2, le bit le plus à droite correspond à $(-2)^0 = 1$, le suivant à $(-2)^1 = -2$, puis $(-2)^2 = 4$, et enfin $(-2)^3 = -8$. En appliquant cette logique, on peut calculer les valeurs décimales de tous les nombres à 4 bits.

Par exemple pour 1001_{-2} : $1 \cdot (-2)^3 + 0 \cdot (-2)^2 + 0 \cdot (-2)^1 + 1 \cdot (-2)^0 = -7$

On fait cela de 0000_{-2} à 1111_{-2} :

Base -2	Décimal
0000	0
0001	1
0010	-2
0011	-1
0100	4
0101	5
0110	2
0111	3
1000	-8
1001	-7
1010	-10
1011	-9
1100	-4
1101	-3
1110	-6
1111	-5

Pour construire la table d'addition en base -2, il faut comprendre que chaque bit ne peut prendre que deux valeurs : 0 ou 1. On fait l'addition bit à bit comme en binaire classique, mais on tient compte de la retenue négative.

$A + B$	Résultat	Retenue
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	-1

$1 + 1 = 0$ avec retenue -1 (car $2 = 0 + (-2) \cdot (-1)$). Les autres additions ($0+0$, $0+1$, $1+0$) restent simples, retenue $= 0$.

Conclusion

Cet exercice demandait de traduire en décimal les seize valeurs entre $(0000)_{-2}$ et $(1111)_{-2}$, puis de construire la table d'addition en base -2 . Pour la première partie, j'ai multiplié chaque bit par la puissance correspondante de la base et additionner le tout. Pour la deuxième partie, j'ai analysée les cas possibles bit à bit en tenant compte de la retenue négative de cette base.

Ce que j'ai compris / appris

Ce qui était nouveau pour moi dans cet exercice c'est de travailler avec une base -2 , où les puissances alternent entre valeurs positives et négatives.

J'ai compris que :

- la méthode de conversion reste la même que pour n'importe quelle base, même quand la base est négative : on multiplie chaque chiffre par la puissance correspondante et on somme ;
- les puissances de -2 alternent entre positif et négatif ($1, -2, 4, -8 \dots$), ce qui explique pourquoi les valeurs décimales obtenues ne suivent pas un ordre croissant comme en binaire classique ;
- pour la table d'addition, le seul cas particulier est $1 + 1$ qui donne 0 avec une retenue de -1 , car $2 = 0 + (-2) \times (-1)$ et les autres cas restent identiques à une addition binaire classique.

3.10 Une Seconde Série d'Exercices...

Pour ce chapitre trois, vous devez rendre les exercices **3.6** et **3.14**. L'exercice **3.15** est pour un Bonus.

Exercice 3.14 (A Rendre)

Étant donné deux nombres représentés en complément à 2, comment peut-on savoir lequel est le plus grand en n'utilisant que des comparaisons de bits entre eux ?

Écrire un programme dans le langage de votre choix pour effectuer cette comparaison. (Avec ou sans saisie de ces nombres par l'utilisateur selon vos connaissances dans le langage choisi)

Pour vous aider : Si les bits de signe sont différents, c'est le nombre où il vaut 0 qui est le plus grand...

Réponse :

On cherche à déterminer lequel de deux nombres représentés en complément à 2 est le plus grand, uniquement en comparant leurs bits. En complément à 2, le bit le plus à gauche, appelé bit de signe, indique si le nombre est positif (0) ou négatif (1).

Le programme commence par convertir les deux nombres en binaire sur un nombre suffisant de bits afin d'obtenir leur représentation complète. Cela permet de gérer correctement à la fois les nombres positifs et négatifs. Ensuite, il récupère le bit de signe de chaque nombre.

Si les bits de signe sont différents, le nombre dont le bit de signe vaut 0 est forcément le plus grand, car il est positif. Si les deux nombres ont le même signe, le programme compare les bits restants un par un, de gauche à droite. Pour les nombres positifs, un bit 1 est plus grand qu'un bit 0 . Pour les nombres négatifs, c'est l'inverse : plus la valeur binaire est grande, plus le nombre réel est petit.

Enfin, si tous les bits sont identiques, les deux nombres sont égaux.

Au début, j'ai rencontrée des difficultés liées au nombre de bits à utiliser, car tous les nombres ne peuvent pas être représentés sur seulement 2 bits par exemple. J'ai également dû chercher comment gérer correctement les nombres négatifs pour éviter que le programme affiche des résultats incohérents, comme dire que -3 est plus grand que -2 .

Au final, j'ai fait l'inverse de ce que je faisais au départ. Avant, je demandais d'abord à l'utilisateur le nombre de bits, puis les nombres a et b . Maintenant, je demande directement les nombres a et b , et ensuite mon

programme calcule automatiquement le nombre minimum de bits nécessaires pour représenter correctement ces deux nombres. Pour cela, j'ai utilisé les fonctions `max` et `abs`, que j'ai apprises l'année dernière dans mes cours de NSI (Numérique et Sciences Informatiques) en terminale mais je pouvais pas les utiliser sans les reprogrammer.

Voici le programme (également disponible dans l'annexe voir fichier README) :

```
1 # Saisie des nombres en decimal
2 a = int(input("Entrez le premier nombre : "))
3 b = int(input("Entrez le deuxieme nombre : "))
4
5 # Calcul du nombre minimum de bits pour representer les deux nombres
6 max_val = max(abs(a), abs(b))
7 bits = 1
8 while (1 << (bits - 1)) <= max_val:
9     bits += 1
10
11 print(f"Nombre de bits utilise : {bits}")
12
13 # Fonction pour convertir un nombre en complement a 2 sur 'bits' bits
14 def to_complement2(n, bits):
15     if n < 0:
16         n = (1 << bits) + n # ajouter 2^bits si le nombre est negatif
17     return n
18
19 # Conversion des nombres en complement a 2
20 a_bin = to_complement2(a, bits)
21 b_bin = to_complement2(b, bits)
22
23 # Determiner le bit de signe (0 = positif, 1 = negatif)
24 def bit_signe(n, bits):
25     if n >= (1 << (bits - 1)):
26         return 1
27     else:
28         return 0
29
30 signe_a = bit_signe(a_bin, bits)
31 signe_b = bit_signe(b_bin, bits)
32
33 # Comparaison selon le signe
34 if signe_a != signe_b:
35     if signe_a == 0:
36         print(f"{a} est plus grand que {b}")
37     else:
38         print(f"{b} est plus grand que {a}")
39 else:
40     # Meme signe : comparer les valeurs en complement a 2
41     if a_bin > b_bin:
42         print(f"{a} est plus grand que {b}")
43     elif a_bin < b_bin:
44         print(f"{b} est plus grand que {a}")
45     else:
46         print(f"{a} et {b} sont egaux")
```

Voici quelques tests pour verifier que le programme fonctionne correctement :
Test avec a et b positifs, et $b > a$:

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 3
Entrez le deuxieme nombre : 6
Nombre de bits utilise : 4
6 est plus grand que 3
```

Test avec a et b positifs, et $a > b$:

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 6
Entrez le deuxieme nombre : 3
Nombre de bits utilise : 4
6 est plus grand que 3
```

Test avec a negatif et b positif :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : -2
Entrez le deuxieme nombre : 9
Nombre de bits utilise : 5
9 est plus grand que -2
```

Test avec a positif et b negatif :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 100
Entrez le deuxieme nombre : -4
Nombre de bits utilise : 8
100 est plus grand que -4
```

Test avec a et b negatifs :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : -4
Entrez le deuxieme nombre : -93
Nombre de bits utilise : 8
-4 est plus grand que -93
```

Test avec a et b egaux :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 94
Entrez le deuxieme nombre : 94
Nombre de bits utilise : 8
94 et 94 sont egaux
```

Conclusion

Cet exercice demandait de comparer deux nombres représentés en complément à 2 en se basant uniquement sur leurs bits. Pour y répondre, j'ai écrit un programme qui commence par calculer le nombre de bits nécessaires pour représenter les deux nombres puis qui compare leur bit de signe avant de comparer les bits restants un par un. Le programme prend en compte les cas suivants : deux nombres positifs, deux nombres négatifs, un positif et un négatif et deux nombres égaux.

Ce que j'ai compris / appris

Cet exercice m'a permis de mieux comprendre comment fonctionne le complément à 2.

J'ai compris que :

- le bit le plus à gauche, appelé bit de signe, détermine si un nombre est positif ou négatif : 0 pour positif, 1 pour négatif ;
- quand deux nombres ont des signes différents, le positif est forcément plus grand ;
- quand deux nombres ont le même signe, la comparaison des bits restants ne fonctionne pas de la même façon selon qu'ils sont positifs ou négatifs : pour les négatifs, une valeur binaire plus grande correspond en réalité à un nombre décimal plus petit.

Exercice 3.15 (Bonus)

Identifier les deux valeurs pour lesquelles le complément à deux ne change pas le bit de signe de la représentation d'un nombre.

Il est important de ne pas oublier d'expliquer et de justifier votre réponse.

Pour vous aider : relire l'encadré "Important" du chapitre.

Réponse :

En complément à 2, le bit de signe (le bit le plus à gauche) indique si un nombre est positif (0) ou négatif (1). D'après l'encadré "Important" du chapitre, toutes les opérations en complément à 2 se font sur un nombre défini de bits, comme 8 bits, 16 bits, etc. Pour que le complément à 2 ne change pas le bit de signe, il faut que le nombre soit 0, soit -1.

En effet, pour 0, sa représentation binaire sur n bits est composée uniquement de 0 et son complément à 2 reste 0, donc le bit de signe reste 0.

Pour mieux comprendre, prenons un exemple : pour 0 sur 4 bits (0000), on fait son complément à deux, c'est-à-dire qu'on inverse tous les bits (1111) puis on ajoute 1 (0000). Donc le bit de signe reste 0.

Pour -1, sa représentation binaire est composée uniquement de 1 et son complément à 2 (qui consiste à inverser tous les bits puis ajouter 1) redonne -1, donc le bit de signe reste 1.

Par exemple, pour -1 sur 4 bits (1111), on fait son complément à deux, on inverse tous les bits (0000), puis on ajoute 1 (1111). Donc le bit de signe reste 1.

Ainsi, les deux valeurs pour lesquelles le complément à 2 ne modifie pas le bit de signe sont 0 et -1, et cette règle est valable quel que soit le nombre de bits utilisé.

Conclusion

Cet exercice demandait d'identifier les deux valeurs pour lesquelles le complément à 2 ne change pas le bit de signe. En s'appuyant sur l'encadré "Important" du chapitre, qui rappelle que toutes les opérations en complément à 2 se font sur un nombre défini de bits, on peut montrer que ces deux valeurs sont 0 et -1 et que cette propriété est valable quel que soit le nombre de bits utilisé.

Ce que j'ai compris / appris

J'ai compris que :

- le complément à 2 d'un nombre consiste à inverser tous ses bits puis à ajouter 1 ;
- pour 0, inverser tous les bits donne une suite de 1, et ajouter 1 redonne une suite de 0 : le bit de signe reste donc 0 ;
- pour -1, inverser tous les bits donne une suite de 0, et ajouter 1 redonne une suite de 1 : le bit de signe reste donc 1 ;
- ces deux cas sont les seuls où le bit de signe est préservé et cette propriété est indépendante du nombre de bits utilisé.

Conclusion du chapitre 3

Ce chapitre portait sur les opérations en binaire et la représentation des nombres signés. La première partie, consacrée à l'addition et à la multiplication en binaire, m'a montré que ces opérations suivent exactement les mêmes règles qu'en base 10 à condition d'utiliser les bonnes tables qui sont d'ailleurs bien plus simples à retenir puisqu'elles ne comportent que deux chiffres.

La deuxième partie, sur la représentation des nombres signés, selon moi, était plus dense. Le complément à 2 est la notion qui m'a demandé le plus d'attention, dans le sens où il fallait comprendre pourquoi le bit de poids fort joue le rôle d'un bit de signe, pourquoi on peut additionner deux nombres signés sans se soucier de leur signe, ou encore pourquoi 0 et -1 sont les deux seules valeurs dont le complément à 2 ne change pas le bit de signe.

Pour résumer les notions essentielles vues dans ce chapitre :

- les tables d'addition et de multiplication en binaire ne comportent que quatre entrées chacune et la seule difficulté est que $1 + 1 = 10$ en binaire, ce qui génère une retenue ;
- un décalage à gauche revient à multiplier un nombre par sa base, un décalage à droite revient à le diviser alors qu'en binaire, cela correspond à des multiplications et divisions par 2 ;
- la représentation la plus courante pour les entiers signés dans un ordinateur est le complément à 2 : on part de la représentation binaire de la valeur absolue, on inverse tous les bits, puis on ajoute 1 ;
- en complément à 2, le bit le plus à gauche joue le rôle de bit de signe : 0 pour un nombre positif, 1 pour un nombre négatif ;
- toutes les opérations en complément à 2 s'effectuent sur un nombre défini de bits.

Dans l'ensemble, c'est un chapitre qui m'a semblé plus exigeant que le précédent mais qui reste abordable.

Sources du chapitre 3

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- Wikipédia. (2023). *Système négabinaire*. https://fr.wikipedia.org/wiki/Syst%C3%A8me_n%C3%A9gabinaire
- Cours de NSI (Terminale) — notions sur le complément à 2 et les comparaisons en binaire.

Chapitre 4 — Représentation des Nombres Flottants et des Caractères

4.2 Une Première Série d'Exercices

Exercice 4.3 (A Rendre)

Une légende dit qu'en guise de récompense, l'inventeur (indien) du jeu d'échecs demande un grain de blé sur la première case de l'échiquier, deux grains sur la seconde, quatre sur la troisième et ainsi de suite jusqu'à la soixante-quatrième.

1. Approximer en base 10 le nombre de grains à poser sur la dernière case.
2. En supposant qu'il faut dix grains de blé pour faire un gramme, approximer le poids que cela représente. (Convertir le résultat en tonnes.)

Réponse :

Les grains de riz évoluent de cette manière :

- 1 grain sur la première case,
- 2 grains sur la deuxième case,
- 4 grains sur la troisième case,
- et ainsi de suite jusqu'à la 64^e case (puisque sur un jeu d'échecs on a 64 cases).

On remarque que chaque case contient le double de la case précédente. On reconnaît ici une suite géométrique de raison 2, où le premier terme est 1.

On cherche le nombre de grains sur la dernière case.

Pour une suite géométrique, le n -ième terme u_n est donné par la formule :

$$u_n = u_1 \times r^{n-1}$$

Dans notre cas :

- $u_1 = 1$ (le premier terme),
- $r = 2$ (la raison),
- $n = 64$ (la dernière case).

Donc le nombre de grains sur la dernière case est :

$$u_{64} = 1 \times 2^{63} = 2^{63}$$

On peut l'approximer en base 10. On sait que :

$$2^{10} \approx 10^3$$

Donc :

$$2^{60} = (2^{10})^6 \approx (10^3)^6 = 10^{18}$$

Puis :

$$2^{63} = 2^3 \times 2^{60} = 8 \times 10^{18} \approx 8 \cdot 10^{18} \text{ grains.}$$

Donc sur la dernière case, il y a environ 8 milliards de milliards de grains.

Il faut 10 grains pour faire 1 gramme.

— Poids en grammes :

$$\text{poids (g)} = \frac{2^{63}}{10} \approx 8 \times 10^{17} \text{ g}$$

— Convertir en kilogrammes : $1 \text{ kg} = 1000 \text{ g}$

$$\text{poids (kg)} = \frac{8 \times 10^{17}}{10^3} = 8 \times 10^{14} \text{ kg}$$

— Convertir en tonnes : $1 \text{ tonne} = 1000 \text{ kg}$

$$\text{poids (tonnes)} = \frac{8 \times 10^{14}}{10^3} = 8 \times 10^{11} \text{ tonnes}$$

Donc :

- Le nombre de grains sur la dernière case est environ 8×10^{18} .
- Le poids correspondant est environ 8×10^{11} tonnes.

Conclusion

Cet exercice demandait d'approximer le nombre de grains sur la dernière case d'un échiquier, en sachant que chaque case contient le double de la précédente, puis d'en déduire le poids correspondant en tonnes. En reconnaissant une suite géométrique de raison 2, on obtient 2^{63} grains sur la dernière case, soit environ 8×10^{18} grains, ce qui correspond à environ 8×10^{11} tonnes.

Ce que j'ai compris / appris

Cet exercice était plutôt accessible car il repose sur des notions de mathématiques que j'ai vu au lycée notamment les suites géométriques. Reconnaître que le nombre de grains double à chaque case permet d'appliquer directement la formule $u_n = u_1 \times r^{n-1}$ et d'obtenir 2^{63} . Ce qui m'a davantage surpris, c'est le résultat final puisqu'on part d'un seul grain et on arrive à un nombre de l'ordre de 10^{18} , je pensais pas que ça pouvait monter si rapidement.

Exercice 4.6 (Bonus)

Si nous nous référons au programme vu au chapitre précédent et rappelé ici :

1. Évaluer l'ordre de grandeur d'un nombre de tours de boucles qu'il effectue. En considérant ici que chaque tour de boucle fait trois opérations : (comparer avec 0, ajouter 1, recommencer la boucle) et que chaque opération prend un tic d'horloge (la fréquence de votre processeur indique le nombre de tics d'horloge par seconde).
2. Évaluer l'ordre de grandeur du temps nécessaire pour exécuter le programme. Placer ensuite ce code du programme (par copier-coller) dans un fichier nommé `foo.c`, le compiler avec la ligne de commande `gcc foo.c`, mesurer son temps d'exécution avec la ligne de commande `time a.out`.
3. Comparer le temps mesuré avec l'estimation "évaluée" de (2). Donnez quelques éléments de justification (c'est le plus important).

Pour vous aider : Quelques recherches à effectuer sur internet vous donneront quelques éléments de réponses...

Rappel du programme :

```
1 int main() {  
2     int t = 1;  
3     while (t != 0) {  
4         t = t + 1;  
5     }  
6     return 0;  
7 }
```

Réponse :

1 : Évaluer l'ordre de grandeur du nombre de tours de boucle

On initialise `t` à 1. La boucle continue tant que `t != 0`. À chaque tour, on ajoute 1 à `t`.

Ainsi :

- `t` est un entier de type `int`, souvent représenté sur 32 bits.
- La valeur maximale pour un entier 32 bits est $2^{31} - 1 = 2\,147\,483\,647$.
- La boucle effectue donc environ 2×10^9 tours avant de s'arrêter.

Nombre d'opérations par tour :

1. Comparer `t != 0`
2. Ajouter 1 à `t`
3. Recommencer la boucle

Donc pour 2×10^9 tours, le programme réalise environ 6×10^9 opérations au total.

2 : Estimation du temps

Chaque opération prend approximativement un tic d'horloge. Le processeur utilisé fonctionne à **3.6 GHz**, soit 3.6×10^9 tics par seconde.

Processeur 12th Gen Intel(R) Core(TM) i7-12700KF (3.60 GHz)

FIGURE 9 – Fréquence du processeur utilisée pour l'estimation (3.6 GHz).

Le temps d'exécution est donc :

$$t = \frac{6 \times 10^9}{3.6 \times 10^9} \approx 1,67 \text{ secondes}$$

Ainsi, le programme met environ **1,7 seconde** à s'exécuter.

3 : Comparer avec le temps mesuré

On vérifie ce résultat :

real 0m1.679s

user 0m1.679s

sys 0m0.000s

Pour répondre à cette question, j'ai utilisé plusieurs sources :

- <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/> (même si elle ne parle pas de code "pur", je trouve que les exemples utilisés illustrent bien le problème)
- <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/> (pour tout ce qui est complexité/nombre d'opérations réelles, temps d'accès, etc.)
- <https://cahier-de-prepa.fr/psi-michelet/download?id=239> (pour la complexité également)

Le temps mesuré pour exécuter le programme est légèrement différent de l'estimation théorique pour plusieurs raisons. Tout d'abord, l'estimation ne prend en compte que le nombre d'opérations de base et la fréquence du processeur, en supposant que chaque opération dure exactement un tic d'horloge. En vérité, le processeur réalise également d'autres opérations telles que la gestion du cache, des interruptions et des processus de fond, ce qui peut avoir un impact sur le temps global. Par ailleurs, notre programme comporte une boucle qui augmente de manière linéaire avec la variable t , et la complexité du programme est directement liée à la dimension du type entier utilisé (en l'occurrence 32 bits). Si nous avions opté pour un nombre plus élevé, la durée aurait été considérablement plus longue.

On peut aussi comparer cette situation à l'estimation du temps dans un projet réel. Une société qui missionne un développeur a besoin de connaître le coût d'une tâche pour faire un devis, mais le temps estimé ne reflète jamais tout le travail réel : coder, réfléchir, organiser, se tromper, recommencer ou se former sont des éléments souvent sous-estimés ou oubliés. De plus, l'estimation varie selon la personne qui réalise la tâche : ce qui prend une journée à un développeur peut en prendre deux à un autre, ou moins à un autre encore. Ainsi, l'estimation n'est jamais parfaitement objective.

Finalement, certains logiciels ou systèmes d'exploitation peuvent occasionner de légères variations dans l'évaluation du temps réel, et la façon dont l'ordinateur accède à la mémoire ou aux registres peut également affecter le temps écoulé. Ainsi, bien que l'évaluation fournisse une approximation correcte, la durée réelle peut varier légèrement en raison de ces éléments matériels, logiciels et humains, ce qui est à la fois normal et prévisible.

Conclusion

Cet exercice demandait d'estimer le nombre de tours de boucle du programme, d'en déduire un temps d'exécution théorique, puis de le comparer au temps mesuré en pratique. En sachant qu'un `int` est représenté sur 32 bits, la boucle effectue environ 2×10^9 tours, soit 6×10^9 opérations au total. Avec un processeur à 3,6 GHz, le temps théorique estimé est d'environ 1,7 seconde ce que le test réel a confirmé avec un temps mesuré de 1,679 seconde.

Ce que j'ai compris / appris

Ce que j'ai trouvée intéressant dans cet exercice, c'est de voir qu'une estimation théorique peut être aussi proche de la réalité. J'ai compris que :

- un `int` est représenté sur 32 bits, ce qui fixe une valeur maximale de $2^{31} - 1$ et explique pourquoi la boucle finit par s'arrêter quand `t` déborde et repasse à 0 ;
- la fréquence du processeur, exprimée en GHz, représente le nombre d'opérations par seconde ;
- le temps mesuré est légèrement différent du temps théorique parce que le processeur effectue en réalité d'autres tâches en parallèle (gestion du cache, interruptions...) qui ne sont pas prises en compte dans l'estimation.

4.10 Une Seconde Série d'Exercices

Vous devez envoyer, pour valider ce quatrième chapitre, les exercices **4.3 (A Rendre)** et **4.13 (A Rendre)**. Pour celles et ceux qui souhaitent un petit plus : le **4.6** est pour un Bonus, l'exercice supplémentaire **4.17** ajoutera un second Bonus.

Exercice 4.13 (A Rendre)

Écrire un programme dans le langage de votre choix pour multiplier deux nombres représentés en base 10 et en virgule flottante entrés par l'utilisateur.

Penser à normaliser le résultat de façon à n'avoir qu'un seul chiffre à gauche de la virgule dans le résultat.

Réponse :

```
1 # Exercice 4.13
2
3 # Donner deux nombres flottants a et b
4 a = float(input("Entrez le premier nombre : "))
5 b = float(input("Entrez le deuxieme nombre : "))
6
7 # Multiplier les deux nombres
8 resultat = a * b
9
10 # Initialiser l'exposant pour apres
11 expo = 0
12
13 # Normaliser pour avoir exactement un chiffre a gauche de la virgule
14 if resultat != 0:
15     while abs(resultat) >= 10:
16         resultat = resultat/10
17         expo = expo + 1
18     while abs(resultat) < 1:
19         resultat = resultat * 10
20         expo = expo - 1
21
22 # Affichage du resultat normalise
23 print("Resultat normalise :", resultat)
24 print("Exposant :", expo)
25 print("Resultat en notation scientifique :", str(resultat) + " x 10^" + str(expo))
```

Pour cet exercice j'ai choisi Python car c'est le langage avec lequel je suis le plus à l'aise pour le moment. Il y a cours de "programmation impérative" en L1 en parallèle qui me permettra à terme de travailler en C, mais j'en suis encore au tout début.

```
1 a = float(input("Entrez le premier nombre : "))
2 b = float(input("Entrez le deuxieme nombre : "))
```

On demande à l'utilisateur de saisir deux nombres. `input()` retourne toujours une chaîne de caractères, donc on utilise `float()` pour convertir directement la saisie en nombre à virgule flottante.

```
1 resultat = a * b
```

On multiplie les deux nombres et on stocke le résultat. À ce stade, le résultat n'est pas encore normalisé ce qui veut dire qu'il peut avoir plusieurs chiffres à gauche de la virgule ou être très petit.

```
1 expo = 0
```

On initialise un exposant à 0. Cet exposant va nous permettre de suivre combien de fois on a multiplié ou divisé le résultat par 10 pour le normaliser.

```
1 if resultat != 0:
2     while abs(resultat) >= 10:
3         resultat = resultat / 10
4         expo = expo + 1
```

Si le résultat est trop grand (supérieur ou égal à 10 en valeur absolue), on le divise par 10 à chaque tour de boucle et on incrémente l'exposant. On utilise `abs()` pour gérer correctement les nombres négatifs car la valeur absolue permet de comparer sans se soucier du signe.

```
1     while abs(resultat) < 1:
2         resultat = resultat * 10
3         expo = expo - 1
```

Si au contraire le résultat est trop petit (inférieur à 1 en valeur absolue), on le multiplie par 10 à chaque tour et on décrémente l'exposant. Ces deux boucles garantissent qu'on obtient toujours exactement un chiffre à gauche de la virgule, ce qui correspond à la normalisation demandée dans l'énoncé.

```
1 print("Resultat normalise :", resultat)
2 print("Exposant :", expo)
3 print("Resultat en notation scientifique :", str(resultat) + " x 10^" + str(expo))
```

On affiche le résultat sous trois formes : la mantisse, l'exposant seul, puis la notation scientifique complète sous la forme $m \times 10^n$. On utilise `str()` pour convertir les nombres en chaînes de caractères afin de pouvoir les concaténer dans le `print`.

Test pour vérifier que tout fonctionne bien (code en annexe) :



Pas de soucis pour un cas "normal".

On teste pour deux très petits nombres $a = 0.001$ et $b = 0.002$. On devrait obtenir 2.0×10^{-6} .

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoi_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : 0.001
Entrez le deuxième nombre : 0.002
Résultat normalisé : 2.0
Exposant : -6
Résultat en notation scientifique : 2.0 x 10^-6
```

FIGURE 10 – Test avec deux très petits nombres.

Le résultat est correct.

Ensuite, on teste un nombre négatif $a = -3.5$ et un nombre positif $b = 2.0$. On devrait obtenir -7.0×10^0 .

```
on_voie_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : -3.5
Entrez le deuxième nombre : 2
Résultat normalisé : -7.0
Exposant : 0
Résultat en notation scientifique : -7.0 x 10^0
```

FIGURE 11 – Test avec un nombre négatif et un nombre positif.

On obtient bien -7.0×10^0 .

On teste ensuite avec $a = 0.0$ et $b = 5.0$.

```
on_voie_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : 0
Entrez le deuxième nombre : 5
Résultat normalisé : 0.0
Exposant : 0
Résultat en notation scientifique : 0.0 x 10^0
```

FIGURE 12 – Test avec zéro.

Le résultat est correct.

Conclusion

Cet exercice demandait d'écrire un programme qui multiplie deux nombres en virgule flottante et normalise le résultat pour n'avoir qu'un seul chiffre à gauche de la virgule. Le programme demande deux nombres à l'utilisateur, les multiplie, puis ajuste le résultat en le divisant ou le multipliant par 10 autant de fois que nécessaire, tout en maintenant un exposant pour garder la trace des ajustements effectués. Le résultat est ensuite affiché sous forme de notation scientifique.

Ce que j'ai compris / appris

La partie qui m'a demandé le plus de réflexion, c'est la normalisation. Au départ je savais ce que je voulais obtenir, un seul chiffre à gauche de la virgule, mais je ne savais pas trop comment l'implémenter. J'ai raisonné de cette façon : pour qu'un nombre ait exactement un chiffre à gauche de la virgule, il doit être compris entre 1 et 10. Donc si le résultat est trop grand, il faut le diviser par 10, et s'il est trop petit, il faut le multiplier par 10. La question qui s'est posée ensuite c'est : combien de fois ? Comme on ne peut pas le savoir à l'avance, un `if` ne suffit pas donc il faut une boucle `while` qui continue tant que le résultat n'est pas dans le bon intervalle. Et à chaque fois qu'on divise ou multiplie par 10, il faut garder trace du nombre d'opérations effectuées, d'où l'idée de l'exposant qu'on incrémente ou décrémenté à chaque tour.

J'ai également compris que :

- si le résultat est supérieur ou égal à 10, on le divise par 10 et on incrémente l'exposant jusqu'à ce qu'il soit dans le bon intervalle ;
- si le résultat est inférieur à 1, on le multiplie par 10 et on décrémenté l'exposant ;

- il faut utiliser `abs()` pour comparer les valeurs en ignorant le signe, sinon un nombre négatif comme -7.0 serait mal géré par les boucles ;
- le cas où le résultat vaut 0 doit être traité séparément pour éviter une boucle infinie.

Exercice 4.17 (Bonus)

Écrire dans le langage de votre choix un programme qui calcule la fonction exponentielle selon la formule de Taylor. Constaté l'erreur d'approximation produite.

Rappel de la formule :

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots + \frac{x^n}{n!}$$

Réponse :

```
1 x = float(input("Entrez la valeur de x : "))
2 n = int(input("Entrez le nombre de termes pour l'approximation (plus grand = plus
  precis) : "))
```

On demande deux valeurs à l'utilisateur : x , la valeur pour laquelle on veut calculer e^x , et n , le nombre de termes de la série de Taylor. On utilise `float()` pour x car il peut avoir une partie décimale, et `int()` pour n car un nombre de termes est forcément un entier, on ne peut pas avoir 7,5 termes.

```
1 exp = 1.0
```

On initialise la somme à 1.0 et pas à 0. Pourquoi ? Parce qu'en regardant la formule de Taylor, le premier terme est toujours 1 (c'est $\frac{x^0}{0!} = 1$). On part donc directement avec ce premier terme et la boucle s'occupera des suivants.

```
1 fact = 1
2 puissance = 1
```

On initialise deux variables en dehors de la boucle. On aurait pu recalculer la factorielle et la puissance depuis zéro à chaque tour, mais ce serait inefficace parce que on aurait calculé x^3 , on aurait recalculé x^1 , x^2 puis x^3 à chaque fois. Ici on garde en mémoire le résultat précédent et on le met à jour à chaque tour c'est plus rapide et plus simple.

```
1 for i in range(1, n+1):
2     puissance = puissance * x
3     fact = fact * i
4     terme = puissance / fact
5     exp = terme + exp
```

À chaque tour de boucle, on calcule le terme suivant de la série de Taylor. À chaque tour, `puissance` est multiplié par x (ce qui donne x^i) et `fact` est multiplié par i (ce qui donne $i!$). On divise ensuite l'un par l'autre pour obtenir le terme $\frac{x^i}{i!}$, qu'on ajoute à la somme.

```
1 print("Approximation de e^", x, "avec", n, "termes : ", exp)
```

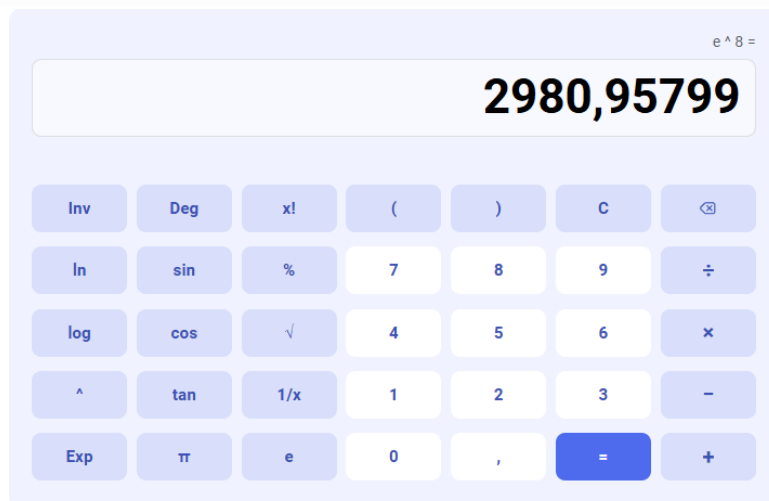
On affiche le résultat final. On reprend x et n dans le message pour que l'affichage soit clair.

Voici le code final :

```
1 # Entrer la valeur de x
2 x = float(input("Entrez la valeur de x : "))
3
4 # Entrer le nombre de termes n pour approximer la somme
5 n = int(input("Entrez le nombre de termes pour l'approximation (plus grand = plus
   precis) : "))
6
7 # Initialisation de la somme avec le premier terme (1)
8 exp = 1.0
9
10 # Boucle pour calculer les termes suivants
11 fact = 1      # Facteur pour le calcul de la factorielle
12 puissance = 1 # x^i
13
14 for i in range(1, n+1):
15     puissance = puissance * x    # Calcul de x^i
16     fact = fact * i              # Calcul de i!
17     terme = puissance / fact    # Terme de la serie
18     exp = terme + exp           # Ajouter le terme a la somme
19
20 # Affichage du resultat approximatif
21 print("Approximation de e^", x, "avec", n, "termes : ", exp)
```

Test pour verifier que le programme fonctionne :

```
>>> %Run 'chap 4 17.py'
Entrez la valeur de x : 8
Entrez le nombre de termes pour l'approximation (plus grand = plus précis) : 6
Approximation de e^ 8.0 avec 6 termes : 934.1555555555556
```



Plus x est grand et/ou n est petit, plus l'erreur sera enorme (par exemple avec $x = 656$ et $n = 4$) :

```
>>> %Run 'chap 4 17.py'
```

```
Entrez la valeur de x : 656
```

```
Entrez le nombre de termes pour l'approximation (plus grand = plus précis) : 4
```

```
Approximation de e^ 656.0 avec 4 termes : 7763477265.0
```



Lorsque la valeur de x augmente les termes de la serie de Taylor devienne plus important ce qui demande une precision et donc davantage de chiffre apres la virgule.

Conclusion

Cet exercice demandait d'implémenter la fonction exponentielle à partir de la formule de Taylor et de constater l'erreur d'approximation produite. Le programme demande à l'utilisateur une valeur x et un nombre de termes n , puis calcule la somme des termes de la série de Taylor de manière progressive. Plus n est grand, plus l'approximation est précise et inversement, plus x est grand avec un n petit, plus l'erreur sera importante.

Ce que j'ai compris / appris

Cet exercice m'a fait découvrir la formule de Taylor, que je ne connaissais pas du tout. Au départ j'ai eu un peu peur, les maths ne sont pas forcément mon point fort, mais une fois la formule comprise, sa a était.

J'ai compris que :

- la série de Taylor permet d'approcher une fonction complexe comme e^x en additionnant des termes de plus en plus petits et que plus on en ajoute, plus on se rapproche de la vraie valeur ;
- l'erreur d'approximation n'est pas un bug. Plus x est grand, plus les termes de la série sont importants et plus il en faut pour obtenir une bonne précision.

Conclusion du chapitre 4

Ce chapitre portait sur la représentation des nombres flottants et des caractères. La première partie sur les ordres de grandeur entre les bases 2 et 10 m'a montré qu'on peut faire des estimations rapides en exploitant le fait que $2^{10} \approx 10^3$ (ex : exercice 4.6 pour estimer le temps d'exécution d'un programme). La partie sur la virgule flottante m'a permis de comprendre comment un ordinateur représente des nombres qui ne sont pas entiers et surtout pourquoi cette représentation est toujours une approximation. Avec un nombre fixe de bits pour la mantisse, certains nombres comme 0.3 ne peuvent tout simplement pas être représentés exactement en base 2. La norme IEEE 754 fixe comment ces bits sont répartis entre le signe, l'exposant et la mantisse et gère des cas particuliers comme l'infini. L'exercice 4.17 sur la formule de Taylor était celui où j'ai eu le plus de difficultés, les maths m'ont fait un peu peur au départ mais j'ai bien compris pourquoi l'erreur augmente quand x est grand et n petit.

Pour résumer les notions principales vues dans ce chapitre :

- $2^{10} \approx 10^3$ est une approximation utilisée pour estimer rapidement des ordres de grandeur en informatique ;
- un nombre en virgule flottante se décompose en un signe, une mantisse et un exposant — la valeur du nombre est $(-1)^s \times m \times 2^e$;
- la représentation en virgule flottante est toujours une approximation : il est impossible de représenter exactement certains nombres comme 0.3 en base 2 avec un nombre fini de bits ;
- la norme IEEE 754 est le standard utilisé par les ordinateurs modernes pour représenter les nombres flottants, sur 32 bits (simple précision) ou 64 bits (double précision).

Sources du chapitre 4

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- OpenClassrooms. (2013). *Combien de bits pour un nombre décimal ?* <https://openclassrooms.com/forum/sujet/combien-de-bits-pour-un-nombre-decimal>
- Apprendre-la-programmation.net. (2020). *Estimer le temps de développement efficacement.* <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/>
- Interstices.info. (2006). *Sciences de l'information : là où le temps sous-tend tant et tant.* <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/>
- Cahier de prépa PSI Michelet. (2021-2022). *Document sur la complexité algorithmique.* <https://cahier-de-prepa.fr/psi-michelet/download?id=239>
- Cours de NSI (Terminale) — suites géométriques, logarithmes, complexité, approximation de Taylor.

Chapitre 5 - Algèbre de Boole

5.4 Exercices

Pour ce cinquième chapitre, les exercices à compléter et à rendre dans le PDF sont : **5.10** (quatre équations au choix), **5.11** (deux tables au choix) et **5.9** pour un **Bonus** (Attention de bien respecter la consigne).

Exercice 5.9 (Bonus)

Imaginons que dans notre langage de tous les jours, nous utilisons l'algèbre de Boole. Que répondrait alors la sage-femme lorsque le nouveau père lui demande : « Alors, c'est un garçon ou une fille ? ».

Dans quel cas la réponse de l'infirmière serait-elle différente si le père lui demandait : « C'est un garçon ou exclusif une fille ? ».

Il est important d'utiliser ici l'ontologie (les mots-métiers) de la sage-femme et non celle du logisticien. La réponse doit être donnée sous forme de phrase naturelle, et non avec une expression logique. Il faut bien expliquer et développer vos réponses.

Réponse :

Quand le papa demande à la sage-femme : "Alors, c'est un garçon ou une fille ?", dans le langage de tous les jours, le mot "ou" veut dire "soit l'un, soit l'autre, mais pas les deux". La sage-femme sait donc instinctivement que le bébé ne peut être ni un garçon ni une fille en même temps et elle répondra simplement : "C'est un garçon" ou "C'est une fille".

En logique, ce "ou" correspond à ce qu'on appelle le "ou exclusif". Si on prenait le "ou" au sens logique inclusif, cela voudrait dire "un garçon ou une fille ou les deux", ce qui n'aurait pas de sens pour un seul bébé, sauf s'il s'agit de jumeaux de sexes différents. Dans ce cas précis, la réponse serait différente : on pourrait alors dire qu'il y a à la fois un garçon et une fille.

Ainsi, dans notre quotidien, on utilise sans s'en rendre compte le "ou exclusif", et ce n'est que dans des cas particuliers comme une naissance de jumeaux que le sens logique du mot "ou" (inclusif) pourrait modifier la réponse.

Conclusion

Cet exercice demandait de réfléchir à la différence entre le "ou" du langage courant et le "ou" en algèbre de Boole, en répondant sous forme de phrases naturelles sans expressions logiques. Dans la vie de tous les jours, on utilise instinctivement le "ou exclusif" sans s'en rendre compte. C'est seulement dans un cas particulier comme une naissance de jumeaux que le "ou inclusif" changerait la réponse.

Ce que j'ai compris / appris

Cet exercice était différent de ce qu'on fait habituellement dans ce chapitre, pas de code, pas d'expression logique, juste des mots et un raisonnement "naturel". Au départ j'ai eu un peu peur de ne pas assez développer, parce que la réponse me semblait évidente une fois qu'on y réfléchit. Mais je pense que justement c'est l'intérêt de l'exercice de montrer que l'algèbre de Boole on l'utilise en permanence dans le langage courant sans s'en rendre compte.

J'ai compris que :

- le "ou" du langage courant correspond en réalité au "ou exclusif" (XOR) de la logique booléenne : on dit "un garçon ou une fille" en sous-entendant que les deux sont impossibles en même temps ;
- le "ou inclusif" de la logique booléenne, lui, accepte que les deux conditions soient vraies simultanément, ce qui dans ce contexte ne ferait sens que pour des jumeaux de sexes différents.

Exercice 5.10 (À rendre)

Simplifier algébriquement les expressions algébriques de gauche, jusqu'à obtenir l'expression de droite.

Choisissez quatre équations ci-dessous selon le degré de difficulté souhaité. Gardez bien l'ordre alphabétique des équations choisies. Par exemple : (a, c, g, i).

b.

$$f = (a \cdot c + b \cdot \bar{c}) \cdot (\bar{a} + \bar{c}) \cdot b$$

Étapes de simplification :

$$\begin{aligned} (a \cdot c + b \cdot \bar{c}) \cdot (\bar{a} + \bar{c}) &= a \cdot c \cdot \bar{a} + a \cdot c \cdot \bar{c} + b \cdot \bar{c} \cdot \bar{a} + b \cdot \bar{c} \cdot \bar{c} && \text{(distribution)} \\ &= 0 + 0 + b \cdot \bar{c} \cdot \bar{a} + b \cdot \bar{c} && \text{(car } a \cdot \bar{a} = 0 \text{ et } c \cdot \bar{c} = 0) \\ &= b \cdot \bar{c} \cdot (\bar{a} + 1) && \text{(factorisation)} \\ &= b \cdot \bar{c} && \text{(puisque } \bar{a} + 1 = 1) \end{aligned}$$

c.

$$f = a \cdot \bar{b} \cdot \bar{c} + a \cdot b \cdot \bar{c} + a \cdot b \cdot c$$

Étapes de simplification :

$$\begin{aligned} f &= a \cdot (\bar{b} \cdot \bar{c} + b \cdot \bar{c} + b \cdot c) && \text{(factorisation par } a) \\ &= a \cdot ((\bar{b} + b) \cdot \bar{c} + b \cdot c) && \text{(factorisation par } \bar{c} \text{ dans les deux premiers termes)} \\ &= a \cdot (\bar{c} + b \cdot c) && \text{(car } \bar{b} + b = 1) \\ &= a \cdot (b + \bar{c}) && \text{(réécriture pour simplifier l'expression)} \end{aligned}$$

e.

$$f = (a + b) \cdot (\bar{a} + b)$$

Étapes de simplification :

$$\begin{aligned} f &= a \cdot \bar{a} + a \cdot b + b \cdot \bar{a} + b \cdot b && \text{(distribution)} \\ &= 0 + a \cdot b + b \cdot \bar{a} + b && \text{(car } a \cdot \bar{a} = 0 \text{ et } b \cdot b = b) \\ &= a \cdot b + b \cdot (\bar{a} + 1) && \text{(factorisation par } b) \\ &= a \cdot b + b && \text{(car } \bar{a} + 1 = 1) \\ &= b && \text{(absorption : } a \cdot b + b = b) \end{aligned}$$

f.

$$f = (x \cdot \bar{y} + z) \cdot (x + \bar{y}) \cdot z$$

Étapes de simplification :

$$\begin{aligned} (x \cdot \bar{y} + z) \cdot (x + \bar{y}) &= x \cdot \bar{y} \cdot x + x \cdot \bar{y} \cdot \bar{y} + z \cdot x + z \cdot \bar{y} && \text{(distribution)} \\ &= x \cdot \bar{y} + 0 + z \cdot x + z \cdot \bar{y} && \text{(car } x \cdot x = x, \bar{y} \cdot \bar{y} = \bar{y}) \\ &= x \cdot \bar{y} + z \cdot (x + \bar{y}) && \text{(factorisation par } z) \\ f &= (x \cdot \bar{y} + z \cdot (x + \bar{y})) \cdot z && \\ &= (x + \bar{y}) \cdot z && \text{(simplification finale)} \end{aligned}$$

Conclusion

Cet exercice demandait de simplifier algébriquement quatre expressions booléennes en appliquant les règles de l'algèbre de Boole.

Ce que j'ai compris / appris — Exercice 5.10

Cet exercice m'a semblé globalement accessible, la simplification algébrique, c'est quelque chose que je trouve pas très difficile. Ce qui m'a posé le plus de problèmes, ce ne sont pas les règles en elles-mêmes mais des erreurs bêtes d'inattention : oublier un terme, mal distribuer et me retrouver avec un résultat faux sans comprendre pourquoi au premier coup d'œil. Au final je m'en suis sorti, mais ça m'a rappelé qu'aller vite est souvent une mauvaise idée.

J'ai compris que :

- $a \cdot \bar{a} = F$ et $a + \bar{a} = V$ (complémentarité) ;
- $a \cdot V = a$ et $a + F = a$ (élément neutre) ;
- $V + a = V$ et $F \cdot a = F$ (élément absorbant) ;
- $a \cdot b + a = a$ et $a + a \cdot b = a$ (absorption) ;
- la distribution : $a \cdot (b + c) = a \cdot b + a \cdot c$ et $a + (b \cdot c) = (a + b) \cdot (a + c)$.

Exercice 5.11 (À rendre)

Utilisez les tables de Karnaugh suivantes pour obtenir la forme algébrique simplifiée de la fonction qu'elles décrivent.

Choisissez deux tables ci-dessous parmi les quatre. Comme pour l'exercice précédent, gardez bien l'ordre alphabétique des tables choisies. Par exemple : (b, d).

Table (a)

La table (a) est :

$cd \backslash ab$	FF	FV	VV	VF
FF	V	V	V	V
FV	V			V
VV	V			V
VF	V	V	V	V

On remarque que toutes les cases de la colonne FF et de la colonne VF contiennent V sur toutes les lignes. Cela signifie que la fonction est vraie pour toutes les valeurs de a et c , et ces V peuvent être couverts par le terme le plus simple qui inclut b et d . Ainsi, la fonction simplifiée s'écrit :

$$f_a(a, b, c, d) = b + d$$

Table (c)

La table (c) est :

$cd \backslash ab$	FF	FV	VV	VF
FF		V	V	
FV	V	V		
VV	V	V	V	V
VF		V	V	

Pour simplifier cette table, on identifie trois groupements de cases V :

- La ligne VV entière (les quatre cases où $c = V$ et $d = V$) forme un rectangle de quatre cases, ce qui donne le terme $c \cdot d$;

- Les deux cases de la colonne FF sur les lignes FV et VV (où $a = F$ et $d = V$) forment un rectangle de deux cases, ce qui donne le terme $\bar{a} \cdot d$;
- Les deux cases des colonnes FV et VV sur les lignes FF et VF (où $b = V$ et $d = F$) forment un rectangle de deux cases, ce qui donne le terme $b \cdot \bar{d}$.

En combinant ces trois termes, on obtient l'expression simplifiée :

$$f_c(a, b, c, d) = c \cdot d + \bar{a} \cdot d + b \cdot \bar{d}$$

Conclusion

Cet exercice demandait d'utiliser les tables de Karnaugh pour obtenir la forme algébrique simplifiée de deux fonctions logiques. La fonction simplifiée est ensuite la réunion de tous ces termes.

Ce que j'ai compris / appris

Les tables de Karnaugh étaient une notion complètement nouvelle pour moi. La méthode m'a semblé mitigée dans le sens où d'un côté, une fois qu'on a compris le principe des groupements, c'est assez mécanique, automatiquement on cherche les rectangles les plus grands possibles et on en déduit les termes. De l'autre, identifier les bons groupements n'est pas toujours évident et il m'est arrivé de me tromper dans la lecture de la table, ce qui m'a donné un résultat faux notamment pour la table (c).

J'ai compris que :

- les groupements doivent toujours contenir un nombre de cases qui est une puissance de 2 (1, 2, 4, 8...);
- il faut maximiser la taille des groupements pour obtenir l'expression la plus simple possible, un grand rectangle élimine plus de variables qu'un petit ;
- une variable est éliminée du terme quand elle change d'état à l'intérieur du groupement ; seules les variables qui restent constantes apparaissent dans le terme final ;
- une même case V peut appartenir à plusieurs groupements différents, ce qui est souvent nécessaire pour couvrir toutes les cases.

Conclusion du chapitre 5

Pour être honnête, ce chapitre est celui que j'ai le moins appréciée. Il y a beaucoup de maths et les maths ne sont pas vraiment mon point fort. Cela dit, le cours est bien expliquée et la progression est suffisamment claire pour que ce soit accessible même sans être à l'aise avec les formules.

Pour résumer les notions importantes vues dans ce chapitre :

- les trois fonctions de base sont **et** ($a \cdot b$), **ou** ($a + b$) et **non** ($\bar{a}$) ;
- le **ou** du langage courant correspond au **ou exclusif** (XOR) de la logique booléenne et non au **ou inclusif** ;
- pour simplifier algébriquement, on applique les règles de base : idempotence ($a \cdot a = a$), complémentarité ($a \cdot \bar{a} = F$), absorption ($a \cdot b + a = a$), dominance ($V + a = V$) et distribution ;
- les tables de Karnaugh permettent de simplifier une expression en regroupant les cases V en rectangles de 2^n cases et plus le rectangle est grand, plus le terme résultant est simple ;
- entre deux cases voisines dans une table de Karnaugh, une seule variable change de valeur.

Sources de tous les chapitres

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- Ifrah, G. (1998). *Histoire universelle des chiffres* (tome 1, p. 454). Robert Laffont.
- Python-3.com. (s. d.). *La méthode .upper()*. <https://fr.python-3.com/?p=2788>
- BornToCode.fr. (2016). *LaTeX : comment insérer et customiser du code source*. <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>
- WolframAlpha. (s. d.). *WolframAlpha : Computational intelligence*. <https://www.wolframalpha.com/>
- Wikipédia. (2023). *Discussion : Numération romaine*. https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine
- Wikipédia. (2023). *Système négabinaire*. https://fr.wikipedia.org/wiki/Syst%C3%A8me_n%C3%A9gabinaire
- OpenClassrooms. (2013). *Combien de bits pour un nombre décimal ?* <https://openclassrooms.com/forum/sujet/combien-de-bits-pour-un-nombre-decimal>
- Apprendre-la-programmation.net. (2020). *Estimer le temps de développement efficacement*. <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/>
- Interstices.info. (2006). *Sciences de l'information : là où le temps sous-tend tant et tant*. <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/>
- Cahier de prépa PSI Michelet. (2021-2022). *Document sur la complexité algorithmique*. <https://cahier-de-prepa.fr/psi-michelet/download?id=239>
- Cours de NSI (Terminale) — représentation des nombres (binaires, complément à 2, flottants), suites, logarithmes, complexité algorithmique.